

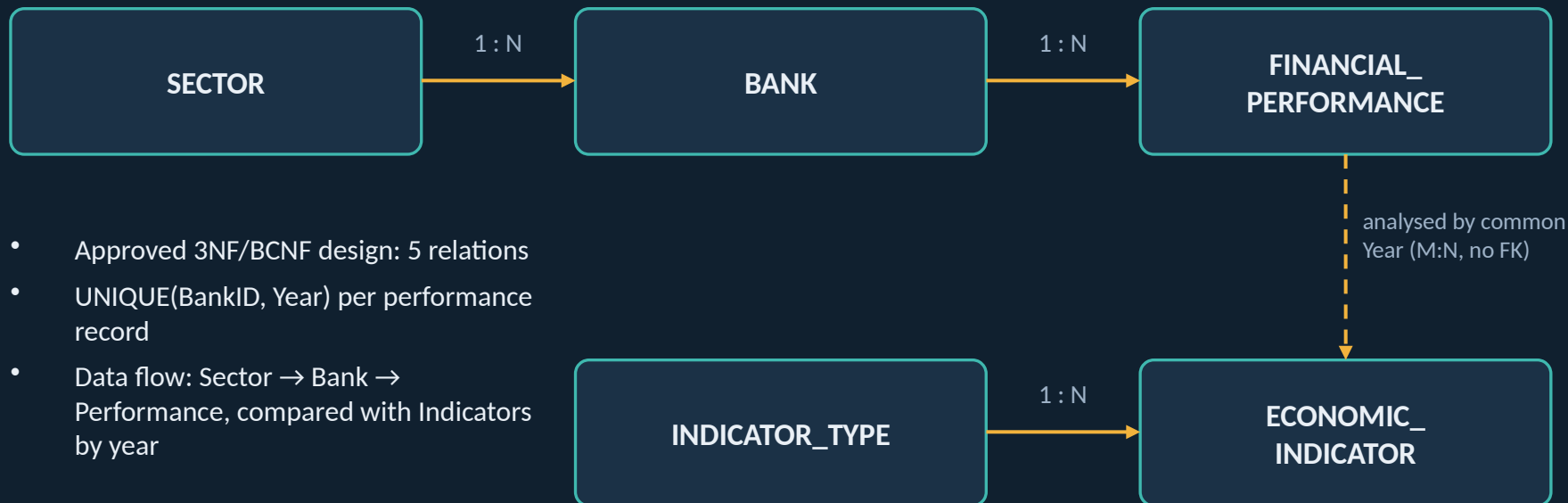
Bank Performance and Economic Indicator Database Management System

TAE-2 Database Implementation & Advanced SQL

Gaurishankar Yadav (P16) | TY B.Tech CSE (Data Science)

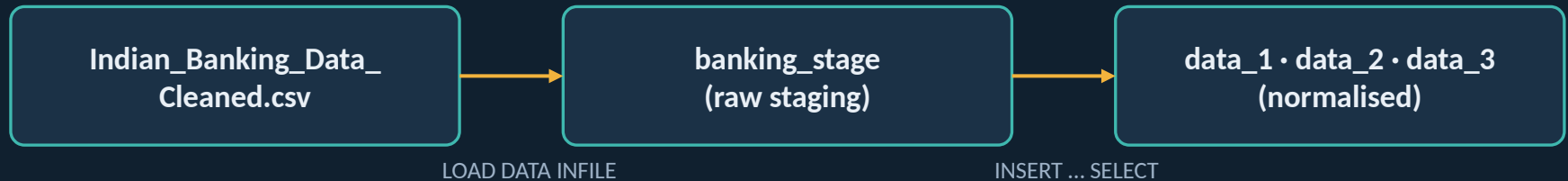
1. Domain & TAE-1 Recall

Indian banking performance linked with macroeconomic indicators (2005–2020)



2. System Architecture

Target RDBMS, tools and load pipeline



Setup

- MySQL 8.0, InnoDB engine, utf8mb4
- MySQL Workbench for DDL, DML, EXPLAIN
- Database: indian_banking_tae2
- Script files: schema, data, join, correlated, stored, trigger, views

TAE-1 → implemented tables

- SECTOR + BANK → data_1 (sector column)
- FINANCIAL_PERFORMANCE → data_3
- INDICATOR_TYPE + ECONOMIC_INDICATOR → data_2 (one row per year)

3. Database Instantiation

DDL and integrity constraints

DDL & keys

- CREATE DATABASE / CREATE TABLE / ALTER TABLE
- PK: data_1(bank_id AUTO_INCREMENT), data_2(report_year), data_3(bank_id, report_year)
- data_3.bank_id → data_1: ON DELETE CASCADE, ON UPDATE CASCADE
- data_3.report_year → data_2: ON DELETE RESTRICT, ON UPDATE CASCADE

Constraints

- NOT NULL on all measures
- UNIQUE(bank_name)
- DEFAULT TRUE on is_active
- CHECK sector IN (Foreign, Private, Public)
- CHECK repo_rate >= 0, total_staff >= 0
- CHECK ratios BETWEEN 0 AND 100

4. Data Population

Transformed raw dataset loaded through a staging table

106

banks (data_1)

16

reporting years (data_2)

1,367

bank-year records (data_3)

- Source: Indian_Banking_Data_Cleaned.csv, 2005–2020, 3 sectors
- Bank sector taken from each bank's latest year; year-level indicators grouped per year
- Joins and filters return non-trivial results across all three tables

5. Complex SQL: Joins

Multi-table inner, outer and self joins (join.sql)

Inner join (3 tables)

data_1 ⋈ data_3 ⋈ data_2: bank, sector, ROA and GDP growth for 2020, ranked by ROA

Left outer join

All banks with their 2020 ROA, including banks without a record that year

Self join

data_1 joined to itself on sector with a.bank_id < b.bank_id to list peer-bank pairs

Full outer join

LEFT JOIN UNION anti-join, since MySQL has no FULL OUTER JOIN

6. Correlated Subqueries & Aggregation

correlated.sql and GROUP BY / HAVING

1

Above yearly average

ROA $\geq$ AVG(ROA) of the same report_year

2

Top NPA per sector-year

provision_npa $\geq$ MAX for that sector and year

3

Above own average

ROA $\geq$ the bank's own long-term average ROA

4

Sector-year summary

GROUP BY sector, year: COUNT, AVG(ROA), MAX(NPA), HAVING
COUNT(*) $\geq$ 1

7. Stored Procedure

Parameterised reporting procedure (stored.sql)

```
CREATE PROCEDURE
  sp_bank_performance_report(
    IN p_sector VARCHAR(20),
    IN p_start_year SMALLINT,
    IN p_end_year SMALLINT)
BEGIN
  IF p_start_year > p_end_year THEN
    SIGNAL SQLSTATE '45000' ...;
  END IF;
  SELECT ... FROM data_1 JOIN data_3
  WHERE sector = p_sector
  AND report_year BETWEEN
    p_start_year AND p_end_year;
END
```

- Returns sector, bank, year, ROA, NPA and loan ratio
- Validates the year range with SIGNAL
- Reusable without rewriting the query

```
CALL sp_bank_performance_report(
  'Private', 2015, 2020);
```

8. Trigger & Dynamic Auditing

trg_data3_audit: AFTER UPDATE ON data_3

1

UPDATE on data_3 fires the trigger

2

Compare OLD.roa with NEW.roa

3

If changed, INSERT into data3_audit

Audit columns: audit_id, bank_id, report_year, old_roa, new_roa, action_name, changed_at

```
IF OLD.roa <> NEW.roa THEN
  INSERT INTO data3_audit
    (bank_id, report_year,
     old_roa, new_roa,
     action_name)
  VALUES (NEW.bank_id,
           NEW.report_year,
           OLD.roa, NEW.roa,
           'UPDATE');
END IF;

-- demo inside a transaction,
-- then ROLLBACK
```

9. Business Reporting Views

Two virtual views (views.sql)

vw_data3_risk_report

- Joins data_1, data_3 and data_2
- Shows ROA, NPA, 5-year ROA risk, loan ratio, GDP growth
- risk_band: HIGH if roa_risk_5yr ≥ 3 or NPA ≥ 0.5 ; MEDIUM if ≥ 2 or ≥ 0.25 ; else NORMAL

vw_sector_year_summary

- GROUP BY sector and year
- bank_count, average ROA, average loan ratio, maximum NPA, GDP growth
- Sector-level trend reporting

CREATE VIEW ... AS SELECT (virtual views, not materialised)

10. Performance Tuning & Indexing

EXPLAIN before and after indexing

Test 1: single-column

```
WHERE roa_risk_5yr >= 3
```

```
CREATE INDEX idx_data3_roa_risk  
ON data_3 (roa_risk_5yr);
```

Test 2: composite

```
WHERE report_year = 2020  
AND provision_npa >= 0.5
```

```
CREATE INDEX idx_data3_year_npa  
ON data_3 (report_year,  
           provision_npa);
```

Flow: DROP existing index → EXPLAIN → CREATE INDEX → EXPLAIN

Compare EXPLAIN columns: type, key, rows, Extra